
Sparkle: LLM-Driven Formal Hardware Synthesis from Natural Language to Silicon

Anonymous Author(s)

Affiliation

Address

email

Abstract

1

2 1 Introduction

3 2 Related Work

4 3 Method

5 We present SPARKLE, an end-to-end framework that translates natural language hardware specifica-
6 tions into formally verifiable, synthesizable hardware designs. The key insight is that by targeting
7 a dependently-typed hardware description language embedded in Lean 4 rather than generating Veri-
8 log directly, the LLM agent receives rich compile-time feedback—type errors, width mismatches,
9 and formal verification failures—that guides iterative refinement toward correct-by-construction
10 hardware.

11 3.1 Overview

12 Figure ?? illustrates the complete pipeline. Given a natural language specification S and an optional
13 reference Verilog implementation $\mathcal{V}_{\text{ref}}$, our system produces a formally described hardware design
14 through the following stages:

- 15 1. **LLM Code Generation.** A coding agent $\mathcal{A}$ translates S into Sparkle HDL (Lean 4) code $\mathcal{L}$,
16 using few-shot examples from a verified benchmark library and iterative error correction.
- 17 2. **Compilation & Extraction.** The Lean 4 compiler type-checks $\mathcal{L}$ and extracts synthesizable
18 SystemVerilog $\mathcal{V}$ via the `#synthesizeVerilog` metaprogram.
- 19 3. **Functional Verification.** $\mathcal{V}$ is simulated against reference testbenches to verify functional cor-
20 rectness.
- 21 4. **Physical Synthesis.** $\mathcal{V}$ is synthesized and placed-and-routed using OpenROAD with the SkyWa-
22 ter 130nm PDK, yielding PPA (Power, Performance, Area) metrics.
- 23 5. **PPA Optimization.** The agent receives PPA feedback and iteratively optimizes the architecture
24 while preserving functional correctness.

25 3.2 Sparkle HDL: Hardware in Lean 4

26 Sparkle HDL is a domain-specific language embedded in Lean 4 that describes hardware using
27 Signal combinators. A signal `Signal dom  $\alpha$` represents a synchronous stream of values of type α

28 under clock domain dom. The embedding in Lean 4 provides three properties that make it an ideal
29 LLM target:

30 **Type-Safe Bit Widths.** Lean 4’s dependent type system encodes bit widths at the type level via
31 `BitVec N`. Width mismatches—a pervasive source of silent bugs in Verilog—become compile-
32 time errors. For example, adding a 4-bit and an 8-bit signal without explicit extension is rejected by
33 the type checker, providing immediate feedback to the LLM agent.

34 **Structural Safety.** Combinational logic forms a directed acyclic graph enforced by the
35 `Signal` monad; state feedback is only possible through explicit registers (`Signal.register`,
36 `Signal.loop`), making combinational loops impossible by construction. Lean’s exhaustive pattern
37 matching prevents unintended latches—a common synthesis hazard in Verilog’s `always` blocks.

38 **Formal Verification.** Designers (or the LLM agent) can state and prove theorems about the hard-
39 ware directly in Lean 4. For instance, an equivalence proof between a specification and an optimized
40 implementation:

```
41 theorem equiv (input : Signal dom (BitVec 3)) :  
42   optimized input = spec input := by  
43   unfold optimized spec; rfl
```

44 This enables the agent to not only generate hardware but also produce machine-checked correctness
45 certificates.

46 **Verilog Extraction.** The `#synthesizeVerilog` metaprogram compiles the Lean 4 hardware de-
47 scription through Sparkle’s intermediate representation (IR) into clean, human-readable SystemVer-
48 ilog. A built-in Design Rule Check (DRC) pass enforces backend-friendly RTL conventions (e.g.,
49 registered outputs) before emission, reducing physical design iterations.

50 3.3 LLM Coding Agent

51 The coding agent $\mathcal{A}$ is an LLM equipped with tool-use capabilities that iteratively writes, compiles,
52 and debugs Sparkle HDL code. We instantiate $\mathcal{A}$ using Claude with the following components:

53 **System Prompt.** The agent receives a structured system prompt containing: (1) the Sparkle HDL
54 syntax reference, including the `Signal` API, type system, and core operators; (2) design patterns
55 for combinational logic, sequential circuits, and finite state machines; (3) critical rules encoding
56 common pitfalls (e.g., “use `Signal.mux` instead of `if-then-else`”); and (4) a prescribed workflow
57 for iterative development.

58 **Tool Suite.** The agent has access to seven tools: `bash` (execute shell commands with 120s
59 timeout), `read_file`, `write_file`, `edit_file` (surgical string replacement), `grep`, `glob`, and
60 `list_directory`. These tools enable the agent to explore the codebase, read benchmark examples,
61 write Lean code, invoke the compiler, and analyze error messages.

62 **Few-Shot Retrieval.** Before generating code, the agent reads from a library of 10 verified bench-
63 mark implementations spanning combinational circuits, sequential logic, and FSMs. These serve as
64 in-context examples that demonstrate correct Sparkle HDL patterns and naming conventions.

65 **Iterative Compile-Fix Loop.** The agent follows a multi-turn conversation protocol:

- 66 1. Read the natural language specification and reference Verilog.
- 67 2. Identify similar benchmark examples via `grep/glob`.
- 68 3. Write an initial Lean 4 implementation to `Generated/{prob_id}.lean`.
- 69 4. Invoke `lake build` and parse compiler diagnostics.
- 70 5. If compilation fails, analyze the error (type mismatch, unknown identifier, syntax error), apply a
71 targeted fix via `edit_file`, and retry.
- 72 6. Once compilation succeeds, verify the extracted SystemVerilog.

73 The agent is allowed up to T turns (default $T=40$) per problem. Each turn consists of one LLM
 74 inference followed by zero or more tool invocations.

75 3.4 Automated Evaluation Pipeline

76 Once the agent produces a candidate Lean 4 file, the evaluation pipeline assesses it through a se-
 77 quence of increasingly stringent checks:

- 78 1. **Lean Compilation.** lake build Generated.{prob_id} verifies type correctness and trig-
 79 gers Verilog extraction.
- 80 2. **SystemVerilog Extraction.** The build output is parsed to extract the generated .sv file. A
 81 TopModule wrapper is automatically generated to map Sparkle’s port naming conventions to the
 82 reference testbench interface.
- 83 3. **Lint Check.** Verilator or Icarus Verilog performs static analysis to catch synthesis-incompatible
 84 constructs.
- 85 4. **Functional Simulation.** The design is simulated against the VerilogEval reference testbench
 86 using Icarus Verilog. The testbench applies input vectors and compares outputs cycle-by-cycle,
 87 reporting the number of mismatches out of total samples.
- 88 5. **Logic Synthesis.** Designs passing simulation are synthesized using Yosys targeting the SkyWater
 89 130nm HD standard cell library (sky130_fd_sc_hd) via the OpenROAD Flow Scripts (ORFS)
 90 Docker environment. This yields gate-level netlists and initial PPA estimates.
- 91 6. **Place & Route.** The synthesized netlist undergoes floorplanning, placement, clock tree synthesis,
 92 routing, and finishing in OpenROAD, producing a GDSII layout. Static timing analysis (STA)
 93 reports worst negative slack (WNS) across multiple PVT corners (SS/TT/FF).

94 The pipeline is fully automated and containerized: synthesis and P&R run inside a Docker container
 95 with pre-installed ORFS and the sky130 PDK, ensuring reproducibility.

96 3.5 PPA Optimization Loop

97 For designs that pass functional simulation and synthesis, we employ a closed-loop PPA optimiza-
 98 tion strategy. The agent receives quantitative feedback on the physical characteristics of its design
 99 and iteratively refines the architecture.

100 Let $\mathbf{p}_i = (\text{area}_i, \text{power}_i, \text{WNS}_i, \text{cells}_i)$ denote the PPA vector after iteration i . The optimization
 101 proceeds for K iterations (default $K=3$):

- 102 1. **Feedback Construction.** A structured message is constructed containing the current PPA metrics
 103 $\mathbf{p}_i$, the optimization history $\{\mathbf{p}_0, \dots, \mathbf{p}_i\}$, and guidance on potential optimization strategies (e.g.,
 104 reducing intermediate signals, merging operations, restructuring mux chains).
- 105 2. **Agent Resume.** The agent resumes from its prior conversation state with the PPA feedback
 106 appended, preserving full context of the design decisions made so far. It modifies the Lean 4
 107 source to produce an optimized variant.
- 108 3. **Re-evaluation.** The modified design is re-evaluated through the full pipeline (compilation $\rightarrow$
 109 simulation $\rightarrow$ synthesis $\rightarrow$ P&R).
- 110 4. **Best Selection.** We track the global best PPA vector $\mathbf{p}^*$ across all iterations. An iteration is
 111 considered an improvement if any metric improves by $>5\%$ without any metric degrading by
 112 $>20\%$:

$$\text{improved}(\mathbf{p}^*, \mathbf{p}_{i+1}) = \exists m : \frac{p_m^* - p_{i+1,m}}{|p_m^*|} > 0.05 \wedge \forall m : \frac{p_{i+1,m} - p_m^*}{|p_m^*|} < 0.20 \quad (1)$$

- 113 5. **Rollback.** If simulation fails after an optimization attempt, the Lean source is rolled back to the
 114 pre-iteration version and the loop continues with the next iteration, rather than terminating early.

115 After all K iterations complete, the globally best design (by PPA) is restored. This exhaustive
 116 strategy ensures the agent explores the full optimization budget rather than converging prematurely
 117 after a single non-improving iteration.

118 3.6 Architecture Exploration

119 For designs where the optimization space is large, we support an architecture exploration mode.
120 The agent generates C distinct architectural candidates (default $C=3$), each implementing the same
121 specification with different micro-architectural choices (e.g., parallel vs. serial datapaths, different
122 encoding schemes, tree vs. chain reduction). Each candidate is independently evaluated through the
123 full pipeline, and the best is selected by PPA metrics.

124 When formal verification is enabled, the agent additionally produces equivalence proofs between the
125 original specification and each optimized architecture:

```
126 theorem arch_equiv (input : Signal dom (BitVec N)) :  
127   arch_optimized input = spec input := by  
128   unfold arch_optimized spec; simp [Signal.mux, ...]
```

129 This provides machine-checked guarantees that architectural optimizations preserve functional
130 correctness—a property that simulation alone cannot ensure for all possible inputs.

131 4 Experiments

132 5 Conclusion

133 References